

Automatic adjoint code generation for seismic imaging with Tapenade

Pengliang Yang^a

^aLaoshan Laboratory, Jimo District, Qingdao, 266237, Shandong, China

ARTICLE INFO

Keywords:

Automatic differentiation
Full waveform inversion
Least-squares reverse time migration
Gauss-Newton method
Binomial checkpointing

ABSTRACT

Manual implementation and maintenance of tangent and adjoint wave propagators are major challenges in multiparameter seismic inversion. We present a reproducible workflow using Tapenade source-transformation automatic differentiation (AD) to generate both operators from a single forward modelling routine. Implemented in the open-source SMIwiz framework, the workflow differentiates a staggered-grid viscoacoustic solver, including parameter conversion, absorbing boundaries, attenuation memory variables, and receiver sampling. The generated operators provide derivatives of the discrete forward calculation and support full waveform inversion (FWI), least-squares reverse-time migration (LSRTM), and matrix-free Gauss–Newton Hessian-vector products. Binomial checkpointing controls forward-state storage through a user-selected snapshot budget, while scripted post-processing adapts the generated stack operations to the C array layout. The directional finite-difference tests verify gradients for logarithmic velocity–density and velocity–impedance parameterisations, both including inverse quality factor. All tests exhibit approximately second-order convergence. Using simple benchmark models, FWI code generated by Tapenade yields higher convergence rate than original manual code from SMIwiz. Pipelined Gauss-Newton algorithms are easily constructed by pipelining the LSRTM routines. These results demonstrate a practical route to consistent derivative operators for linearised and nonlinear seismic inversion, with reduced manual adjoint development and explicit runtime–memory tradeoffs.

1. Introduction

Seismic imaging via full-waveform inversion (FWI) estimate subsurface properties by minimizing the mismatch between recorded and simulated seismic data (Virieux and Operto, 2009). At the core of this minimization is the gradient of the data-misfit functional with respect to the model parameters. The adjoint-state method (Chavent and Plessix, 1999; Tarantola, 1984) evaluates this gradient at a cost independent of the number of model parameters, requiring only one forward propagation of the wavefield and one reverse propagation of the adjoint wavefield. This makes it the method of choice for problems with millions of parameters, as routinely encountered in reverse-time migration (RTM) (Farmer, Zhou and Jones, 2009), least-squares RTM (LSRTM) (Nemeth, Wu and Schuster, 1999; Yang and Ji, 2026), and multiparameter FWI (Yang, Brossier, Métivier and Virieux, 2016a; Yang, Brossier, Métivier, Virieux and Zhou, 2018).

Implementing the adjoint of a production wave propagator is, however, a substantial software task. The forward solver typically includes convolutional perfectly matched layers (CPML) (Komatitsch and Martin, 2007), attenuation memory variables (Yang et al., 2016a), and complex acquisition geometry; each of these stateful components must be reversed correctly. One common strategy transforms the continuous adjoint equation into a form that matches the forward equation, allowing the forward modelling engine to be reused (Vigh, Jiao, Watts and Sun, 2014; Yang, 2024). This adjoint-then-discrete (ATD) approach works for many physical regimes but requires manual derivation, does not automatically preserve the accuracy of the discrete forward scheme (Sirkes and Tziperman, 1997; Chavent, 2009), and must be repeated whenever the forward discretisation changes.

Automatic differentiation (AD) offers an alternative. Source-transformation AD applies the chain rule directly to the source code of the forward program, producing a new program that computes the derivative of any specified output with respect to any independent input (Griewank and Walther, 2008; Hascoët and Pascual, 2013). Because it differentiates the discrete implementation—the discrete-then-adjoint (DTA) paradigm—the resulting gradient inherits the truncation accuracy of the forward scheme. AD is complementary to operator-overloading approaches, which can

✉ yangpengliang@lsnl.cn (P. Yang)
ORCID(s): 0000-0002-9068-9344 (P. Yang)

be applied to unstructured codes but incur runtime overhead, and to hand-coded adjoints, which offer full control but are costly to maintain. Source-transformation AD has been successfully applied to large geoscientific codes, most notably the MIT general circulation model (Gaikwad, Narayanan, Hascoët, Campin, Pillar, Nguyen, Hückelheim, Hovland and Heimbach, 2025).

A key practical challenge for AD in seismic imaging is the long time loop. The adjoint requires the forward wavefield trajectory, yet storing every time step is prohibitive. Checkpointing (Griewank, 1992; Griewank and Walther, 2000) provides the standard solution: store selected snapshots and recompute segments between them during the reverse pass. Optimal binomial checkpoint schedules are available in tools such as Revolve (Griewank and Walther, 2000) and have been applied to RTM (Symes, 2007).

Operator-overloading AD frameworks have recently gained attention in seismic inversion. Treating the wave-equation time stepping as a recurrent neural network enables FWI to be implemented directly in PyTorch or TensorFlow, leveraging their built-in gradient engines (Richardson, 2018). Zhu et al. (Zhu, Xu, Darve and Beroza, 2021) developed a general seismic inversion framework (ADSeismic) using reverse-mode AD, and the ADFWI package (Liu, Li, Zou and Li, 2025) provides an open-source, PyTorch-based AD framework with flexible workflow configurations. These approaches benefit from the convenience of operator-overloading AD and GPU acceleration, but the runtime overhead of building and traversing the computational graph at every time step can be significant for large-scale 3D problems with long time loops.

Despite these advances, wave-equation-based seismic imaging methods—RTM, LSRTM, and FWI—remain dominated by hand-derived adjoint implementations in compiled languages. In this paper, we present a systematic workflow based on the source-transformation AD tool Taped. Our contribution is a reproducible engineering framework centered on Taped that:

1. defines a narrow AD boundary around a viscoacoustic CPML propagator through a dedicated forward routine, isolating the active numerical computation from passive infrastructure;
2. generates both the tangent linear model (Born operator) and its transpose (migration/gradient operator) from this single routine, with binomial checkpointing managing the forward-trajectory storage budget;
3. supplies a version-controlled post-processing script that repairs the pointer-to-pointer stack operations produced by Taped’s C back-end;
4. assembles the generated operators into a complete linearised inversion toolbox—Born modelling, migration, and the Gauss–Newton Hessian-vector product—with symmetric wrappers handling parameter conversion and shot stacking;
5. validates the gradient through directional finite-difference tests that confirm second-order convergence for multiparameter families $(\ln v_p, \ln \rho)$, $(\ln v_p, \ln I_p)$, and $\ln Q^{-1}$, and compares FWI convergence against a manually coded adjoint.

The implementation and data are available via the accompanying SMIwiz_visco repository. The contribution is not a new imaging condition or a runtime benchmark, but a reproducible engineering workflow that replaces separately maintained adjoint stencils with automated, version-controlled code generation.

2. Forward modelling: A viscoacoustic instance

We consider the following viscoacoustic wave equation (Yang, Brossier, Métivier and Virieux, 2016b)

$$\begin{cases} \rho \partial_t \mathbf{v} + \nabla p = 0 \\ \partial_t p + \kappa \left(\nabla \cdot \mathbf{v} - \sum_{\ell=1}^L Y_\ell \xi_\ell \right) = f \\ \partial_t \xi_\ell + \omega_\ell \xi_\ell = \omega_\ell \nabla \cdot \mathbf{v}, \quad \ell = 1, \dots, L. \end{cases} \quad (1)$$

Here ρ and κ are the density and bulk modulus, respectively, with $\kappa = \rho V_p^2$. The quantity Q is the quality factor of seismic attenuation, p is the pressure, and $\mathbf{v} = (v_x, v_y, v_z)^\top$ is the particle-velocity vector. The source term is denoted by f . The energy is dissipated by L standard linear solids with memory variables ξ_ℓ , reference angular frequencies ω_ℓ , and reference anelastic coefficients Y_ℓ . Under the assumption $Q \gg 10$, the coefficients can be represented as $Y_\ell = y_\ell Q^{-1}$,

where y_ℓ is obtained by least-squares optimization to fit a constant- Q target over logarithmically spaced frequencies (Yang et al., 2016a). Setting $L = 0$ removes the relaxation term and recovers the purely acoustic propagator.

The wave equation (1) can be compactly written as a linear system

$$\underbrace{\begin{bmatrix} \partial_t & 0 & \rho^{-1}\nabla \\ -\omega\nabla\cdot & \text{diag}(\partial_t + \omega_\ell) & 0 \\ \kappa\nabla\cdot & -\kappa(Y_1, \dots, Y_L) & \partial_t \end{bmatrix}}_{A(m)} \underbrace{\begin{bmatrix} \mathbf{v} \\ \xi \\ p \end{bmatrix}}_u = \underbrace{\begin{bmatrix} 0 \\ 0 \\ f \end{bmatrix}}_s. \quad (2)$$

where $\xi = (\xi_1, \dots, \xi_L)^\top$ and $\omega = (\omega_1, \dots, \omega_L)^\top$, while the model vector m is a composite function of physical parameters ρ, κ, Q^{-1} . The modelled wavefield u can therefore be compactly expressed through an inversion of the linear system, i.e., $u = A^{-1}s$. The synthetic data d_{cal} modelled from the model m is then obtained by extracting the wavefield at a number of receiver positions at every time step through a restriction operator R

$$d_{cal} = Ru = RA^{-1}(m)s. \quad (3)$$

This reveals that the synthetic data d_{cal} has a nonlinear dependence on the model m and a linear dependence on the source s .

3. Seismic imaging: An inversion perspective

3.1. Full waveform inversion

Full waveform inversion is a nonlinear optimization procedure to minimize the data difference between the synthetic data d_{cal} and the observations d_{obs} in the least-squares sense

$$\Phi_{FWI}(m) = \frac{1}{2} \|W(d_{cal} - d_{obs})\|^2, \quad (4)$$

where W is the data weighting matrix. Starting from an initial model m_0 , the minimization is usually performed iteratively through

$$m_{k+1} = m_k + \alpha_k \delta m_k, \quad k = 0, 1, \dots \quad (5)$$

where at the k -th iteration the model increment δm_k satisfies the normal equation

$$H(m_k) \delta m_k = -\nabla \Phi(m_k), \quad (6)$$

where $\nabla \Phi(m_k)$ and $H(m_k) = \nabla^2 \Phi(m_k)$ are the gradient of the misfit and the Hessian matrix anchored around m_k . The size of the Hessian matrix $H(m_k)$ scales quadratically with the dimensionality of the model m_k . Due to the high dimensionality of the problem, it is therefore prohibitively expensive to store it in memory to solve equation 6. In practice, the descent direction δm_k is computed via two different approaches:

- compute $\delta m_k = -B \nabla \Phi$ with B a rank-2 approximation of H^{-1} using stored gradients at previous iterations. In practice, the action of B to $\nabla \Phi$ is computed without forming B explicitly. This is referred to quasi-Newton L-BFGS method.
- solve $H_{GN} \delta m_k = -\nabla \Phi$ using conjugate gradient (CG) method, where H_{GN} is a Gauss-Newton approximation of the full Hessian $H(m_k)$ which still satisfies semi symmetric positive definite (SPD) property. The action of the Gauss-Newton Hessian H_{GN} to a field vector p can be efficiently computed in matrix-free fashion. This method is coined Newton-CG (or truncated Newton) method (Yang et al., 2018).

Both L-BFGS and Gauss-Newton approximations are not scale invariant, the step length α_k in equation 5 is often found by line search. Let us remark that approximating Hessian with identity leads to steepest descent method, which is rather inefficient in nonlinear optimization. Approximation of the descent direction using a linear combination of the descent direction at previous iteration and the current gradient leads to nonlinear conjugate gradient (NLCG) method, which goes beyond the framework of equation 6. In all cases, evaluation of function and the gradient is essential, which

will be conveniently constructed using the AD tool Tapenade shortly. The computation of matrix vector product for Gauss-Newton Hessian can also be accurately automated using Tapenade. This is the key contribution of this work.

3.2. Least-squares reverse time migration

Let $F(m)$ denote the nonlinear modelling operator that maps a model m to the recorded data $d_{\text{cal}} = Ru(m) = F(m)$ of equation (3). Around a background model m_0 , a small perturbation δm therefore has the first-order expansion

$$F(m_0 + \delta m) = F(m_0) + J_0 \delta m + \mathcal{O}(\|\delta m\|^2), \quad (7)$$

where $J_0 = F'(m_0)$ is the Jacobian evaluated along the background trajectory m_0 . Its elements are given by

$$(J_0)_{ij} = \frac{\partial d_{\text{cal},i}}{\partial m_j}, \quad (8)$$

where $i = 1, \dots, N$ is the index of data space, while $j = 1, \dots, M$ is the index of model space.

When a smooth, reliable background model m_0 is available, the synthetic data modelled from this m_0 contains tomographic components which often match the early arrivals of the observed data d_{obs} very well. Subtracting $F(m_0)$ from the observed data, the remaining part

$$\delta d_0 = d_{\text{obs}} - F(m_0) = d_{\text{obs}} - Ru(m_0) \quad (9)$$

will mainly be the reflections generated due to the model perturbation δm , which usually has nonzero values at the interfaces. Dropping the 2nd order dependence of the data on the model perturbation, we then seek for the optimal model perturbation δm by reformulating the FWI objective in the linear regime:

$$\Phi_{\text{LSRTM}}(\delta m) = \frac{1}{2} \|W (J_0 \delta m - \delta d_0)\|^2, \quad (10)$$

This is exactly the linearized waveform inversion proposed by Tarantola (1984), and more often coined least-squares reverse time migration (LSRTM) (Yang and Ji, 2026). The optimality condition, $\nabla_{\delta m} \Phi_{\text{LSRTM}} = 0$, yields

$$J_0^T W^T W J_0 \delta m = J_0^T W^T W \delta d_0. \quad (11)$$

where the right hand side is the conventional RTM image (Farmer et al., 2009; Yang and Ji, 2026). Thanks to the semi-SPD property of the operator $J_0^T W^T W J_0$, conjugate gradient method is routinely used to solve the LSRTM problem efficiently. The application of Jacobian matrix and its transpose (J_0 and J_0^T) are often coined Born modelling and migration, in the context of LSRTM.

3.3. Gauss-Newton as a nested nonlinear-linear iteration

Gauss-Newton does not require assembling a dense Hessian. Let $\delta d = d_{\text{obs}} - F(m)$ and consider the nonlinear FWI objective

$$\Phi_{\text{FWI}}(m) = \frac{1}{2} \|W \delta d\|^2. \quad (12)$$

Its gradient is

$$g(m) = -J(m)^T W^T W \delta d, \quad J(m) = F'(m). \quad (13)$$

For a model-space vector x , the exact Hessian action contains both a Gauss-Newton term and a residual-curvature term:

$$\nabla^2 \Phi(m)x = J^T W^T W J x + [DJ(m)[x]]^T W^T W \delta d. \quad (14)$$

Here $DJ(m)[x]$ is the directional derivative of the Jacobian. The Gauss-Newton approximation drops this second, residual-dependent term, giving the positive-semidefinite action (Santosa and Symes, 1988; Pratt, Shin and Hicks,

1998)

$$H_{\text{GN}}(m)x = J(m)^\top W^\top W J(m)x. \quad (15)$$

This approximation is standard when residuals are sufficiently small or when the neglected curvature is unsuitable for a stable descent step (Virieux and Operto, 2009). At the k -th iteration, the normal equation (6) becomes

$$J_k^\top W^\top W J_k \delta m_k = J_k^\top W^\top W \delta d_k = -g(m_k). \quad (16)$$

The above equation is remarkable because it clearly connects Gauss-Newton method with LSRTM: Gauss-Newton FWI estimates a model perturbation using LSRTM at every iteration, based on the current model m_k . In other words, LSRTM is nothing else than the first iteration of Gauss-Newton FWI with fixed background model m_0 .

4. Implementation and automated workflow with Tapenade

The Jacobian and its transpose—realized as the tangent linear model (TLM) and the adjoint model (ADM) in automatic differentiation—are the core linear operators for LSRTM and Gauss–Newton FWI. Coding them manually is tedious and error-prone, as has been done in SMIwiz open-source software (Yang, 2024). We now present an automated Tapenade workflow that differentiates a single forward modelling routine to produce both operators, together with the checkpoint and post-processing infrastructure needed for production inversions.

4.1. Tapenade-facing forward modelling

Without the source terms, equation (1) can be easily discretized using staggered-grid finite difference method (Yang et al., 2016b):

$$\mathbf{v}^{n+\frac{1}{2}} = \mathbf{v}^{n-\frac{1}{2}} - \Delta t \rho^{-1} \nabla p^n, \quad (17a)$$

$$\xi_\ell^{n+1} = e^{-\omega_\ell \Delta t} \xi_\ell^n + (1 - e^{-\omega_\ell \Delta t}) \nabla \cdot \mathbf{v}^{n+\frac{1}{2}}, \quad (17b)$$

$$p^{n+1} = p^n - \Delta t \kappa \left[\nabla \cdot \mathbf{v}^{n+\frac{1}{2}} - \frac{Q^{-1}}{2} \sum_{\ell=1}^L y_\ell \left(\xi_\ell^n + \xi_\ell^{n+1} \right) \right]. \quad (17c)$$

To mimic the wave propagating to infinitely far distance, convolutional perfectly matched layers (CPML) are padded to the domain of interest.

As sketched in Listing 1, a forward simulation routine first extends velocity $\mathbf{vp}$ and density ρ from the physical domain to the whole domain including CPML, and converts them into bulk modulus κ and buoyancies (bux , buy , buz , which are the inverse of density averaged in x , y and z directions over a staggered grid). Six CPML arrays (memD1p , memD2p , memD3p , memD1vz , memD2vx , memD3vy) are employed to handle the absorbing boundary condition. A forward modelling consisting of nt time steps performs four steps within each time step:

- `fdtd_update_v()`: the particle velocities (vx , vy , vz) are updated according to Equation 17a;
- `fdtd_update_p()`: the attenuation memory variables xi1 , xi2 , xi3 and the pressure p are updated according to Equation 17b and 17c, thanks to the divergence of particle velocity and the inverse of the quality factor qinvmod ;
- `inject_source()`: the contribution of the source term is only added at specific location via Kaiser-windowed sinc interpolation rather than the whole grid. It is therefore split from the above field updates.
- `extract_wavefield()`: The modelled pressure data dcal is also recorded at the receiver locations using Kaiser-windowed sinc interpolation.

```
void modelling(sim_t *sim, acq_t *acq,
    float ***p, float ***vz, float ***vx, float ***vy,
    float ***memD1p, float ***memD2p, float ***memD3p,
    float ***memD1vz, float ***memD2vx, float ***memD3vy,
    float ***xi1, float ***xi2, float ***xi3,
```

```

126         float ***vp, float ***rho, float ***qinv,
127         float ***vpmod, float ***rhomod, float ***qinvmod,
128         float ***kappa, float ***buz, float ***bux, float ***buy,
129         float **dcal)
130 {
131     int it;
132     int nt = sim->nt;
133     int nsnap = sim->nsnap;
134
135     (void)nsnap; /* Referenced by Tapenade's BINOMIAL-CKP directive. */
136     convert_parameters(sim, vp, rho, qinv, vpmod, rhomod, qinvmod,
137                     kappa, buz, bux, buy);
138
139     /* $AD BINOMIAL-CKP nt nsnap 0 */
140     for (it = 0; it < nt; it++) {
141         fdttd_update_v(sim, p, vz, vx, vy, memD1p, memD2p, memD3p,
142                     buz, bux, buy);
143         fdttd_update_p(sim, p, vz, vx, vy, memD1vz, memD2vx, memD3vy,
144                     xi1, xi2, xi3, kappa, qinvmod);
145         inject_source(sim, acq, p, sim->stf[it]);
146         extract_wavefield(sim, acq, p, dcal, it);
147     }
148 }
149

```

Listing 1: Tapenade-facing forward loop.

The above dedicated modelling routine provides a reliable differentiation boundary. The high-level driver should own parameter parsing, allocation, model construction, file I/O, diagnostic reporting, and cleanup. The routine supplied through `-head` should contain the deterministic numerical calculation, make its active inputs and useful outputs explicit, and call only the numerical kernels needed to evaluate those outputs. The passive contexts `sim_t` and `acq_t` carry grid dimensions, time stepping, the wavelet, acquisition geometry, CPML damping, and the standard-linear-solid fit coefficients; they are not active in the differentiation.

4.2. Automated TLM and adjoint program

In the language of automatic differentiation, the tangent linear model (TLM) describes how a small perturbation δm in the parameters produces a small perturbation δu in the solution, to first order. Differentiation of the wave equation $A(m)u = f$ with respect to m in direction δm yields

$$A(m)\delta u = -\frac{\partial A(m)}{\partial m}[\delta m]u. \quad (18)$$

The above is precisely the Born modelling equation in LSRTM (Yang and Ji, 2026). Equation (18) shows that the scattering field δu obeys the same wave equation except for a different source term, which has a linear dependence on the model perturbation δm and background wavefield u . Besides the injection of the source, both forward equation and Born modelling equation follows the same time stepping rule in equation (17), which can be cast as a state recurrence

$$u^{n+1} = \mathcal{L}_n(u_n; m). \quad (19)$$

according to Appendix A. The scattered field is thus advanced via

$$\delta u^{n+1} += \left(\frac{\partial \mathcal{L}_n}{\partial u_n} \right) \delta u^n, \quad (20)$$

where `'+='` accumulates the contributions for $n = 0, \dots, n_t - 1$ during forward time stepping. Tapenade operates on this discrete implementation through the differentiation graph. The Born modelling code is thus obtained by differentiating each stencil of the forward code: the perturbation wavefields obey the identical staggered difference operators with the background coefficients.

The command in Listing 2 calls Taped to generate the Born modelling code `modelling_d()` from the original forward modelling() using the mode flag '-tangent' (or '-d'). Listing 3 shows that the TLM of each substep in modelling() has been automatically generated. The suffix `_d` is appended to distinguish the TLM code from the forward routine.

```
# forward-mode tangent (TLM): modelling_d.c and companions
taped -tangent \
  -head 'modelling(dcal)/(vp_rho_qinv)' \
  -outputdirectory taped_out -I../include \
  modelling.c extend_model.c fdtd.c inject_extract.c
```

Listing 2: Taped invocations for tangent mode.

```
void modelling_d(sim_t *sim, acq_t *acq, float ***p, float ***pd, float ***vz,
float ***vzd, float ***vx, float ***vxd, float ***vy, float ***vyd,
float ***memD1p, float ***memD1pd, float ***memD2p, float ***memD2pd,
float ***memD3p, float ***memD3pd, float ***memD1vz, float ***memD1vzd
, float ***memD2vx, float ***memD2vxd, float ***memD3vy, float ***
memD3vyd, float ***xi1, float ***xi1d, float ***xi2, float ***xi2d,
float ***xi3, float ***xi3d, float ***vp, float ***vpd, float ***rho,
float ***rhod, float ***qinv, float ***qinvd, float ***vpmod, float **
*vpmodd, float ***rhomod, float ***rhomodd, float ***qinvmod, float **
*qinvmodd, float ***kappa, float ***kappad, float ***buz, float ***
buzd, float ***bux, float ***buxd, float ***buy, float ***buyd, float
**dcal, float **dcald) {
    int it;
    int nt = sim->nt;
    int nsnap = sim->nsnap;
    (void)nsnap;
    /* Referenced by Taped's BINOMIAL-CKP directive. */
    convert_parameters_d(sim, vp, vpd, rho, rhod, qinv, qinvd, vpmod, vpmodd,
        rhomod, rhomodd, qinvmod, qinvmodd, kappa, kappad,
        buz, buz, bux, buxd, buy, buyd);
    memset(pd[0][0], 0, sim->n123pad*sizeof(float));
    memset(memD2pd[0][0], 0, sim->n1pad*2*sim->nb*sim->n3pad*sizeof(float));
    memset(memD1vzd[0][0], 0, 2*sim->nb*sim->n2pad*sim->n3pad*sizeof(float));
    memset(xi1d[0][0], 0, sim->n123pad*sizeof(float));
    memset(xi2d[0][0], 0, sim->n123pad*sizeof(float));
    memset(xi3d[0][0], 0, sim->n123pad*sizeof(float));
    memset(memD3vyd[0][0], 0, (sim->n3>1 ? sim->n1pad*sim->n2pad*2*sim->nb : 1)*sizeof(float));
    memset(memD3pd[0][0], 0, (sim->n3>1 ? sim->n1pad*sim->n2pad*2*sim->nb : 1)*sizeof(float));
    memset(dcald[0], 0, sim->nt*acq->nrec*sizeof(float));
    memset(memD2vxd[0][0], 0, sim->n1pad*2*sim->nb*sim->n3pad*sizeof(float));
    memset(vxd[0][0], 0, sim->n123pad*sizeof(float));
    memset(vyd[0][0], 0, sim->n123pad*sizeof(float));
    memset(vzd[0][0], 0, sim->n123pad*sizeof(float));
    memset(memD1pd[0][0], 0, 2*sim->nb*sim->n2pad*sim->n3pad*sizeof(float));
    for (it = 0; it < nt; ++it) {
        if (iproc == 0 && it%100 == 0)
            printf("it-----%d\n", it);
        fdtd_update_v_d(sim, p, pd, vz, vzd, vx, vxd, vy, vyd, memD1p, memD1pd
            , memD2p, memD2pd, memD3p, memD3pd, buz, buz, bux,
            buxd, buy, buyd);
        fdtd_update_p_d(sim, p, pd, vz, vzd, vx, vxd, vy, vyd, memD1vz,
            memD1vzd, memD2vx, memD2vxd, memD3vy, memD3vyd, xi1,
            xi1d, xi2, xi2d, xi3, xi3d, kappa, kappad, qinvmod,
            qinvmodd);
    }
}
```

Table 1

Variables used by Tapenade in forward, adjoint and tangent modes

Forward variable (m, u)	Adjoint variable ($\frac{\partial \Phi}{\partial m}, \frac{\partial \Phi}{\partial u}$)	Tangent variable ($\delta m, \delta u$)
p, vz, vx, vy	pb, vzb, vxb, vyb	pd, vzd, vxd, vyd
memD1p, ..., memD3vy	memD1pb, ..., memD3vyb	memD1pd, ..., memD3vyd
xi1, xi2, xi3	xi1b, xi2b, xi3b	xi1d, xi2d, xi3d
vp, rho, qinv	vpb, rhob, qinvb	vpd, rhod, qinvd
kappa, qinvmod	kappab, qinvmodb	kappad, qinvmodd
buz, bux, buy	buzb, buxb, buyb	buzd, buxd, buyd
dcal	dcalb	dcald

```

217     inject_source_d(sim, acq, p, pd, sim->stf[it]);
218     extract_wavfield_d(sim, acq, p, pd, dcal, dcald, it);
219 }
220 }
221 
```

Listing 3: Tapenade TLM routine

The adjoint of the TLM, i.e., migration, is the same discrete first derivative read in reverse. Let λ^n denote the sensitivity of the scalar objective with respect to the state at step n , i.e., $\lambda^n = \partial \Phi / \partial u^n$. Differentiating the recurrence and transposing gives the one-step discrete adjoint relation

$$\lambda^n += \left(\frac{\partial \mathcal{L}_n}{\partial u_n} \right)^\top \lambda^{n+1}, \quad (21)$$

where '+=' accumulates the contributions for $n = n_t - 1, \dots, 0$ during the reverse sweep. Using the command in Listing 4, Tapenade creates the adjoint of the forward code as `modelling_b()` in Listing 5 by activating the mode flag '-reverse' (or '-b'). The suffix `_b` highlights the backward time direction in reverse mode.

```

225 # reverse (FWI gradient and LSM migration): modelling_b.c and companions
226 tapenade -reverse \
227 -head 'modelling(dcal)/(vp_rho_qinv)' \
228 -outputdirectory tapenade_out -I../include \
229 modelling.c extend_model.c ftdt.c inject_extract.c
230
231 
```

Listing 4: Tapenade invocations for reverse mode.

In the implementation, the Born/migration pair is therefore

$$\begin{aligned} \text{modelling_d}() : \quad x &\mapsto y = J_0 x, \\ \text{modelling_b}() : \quad \bar{y} &\mapsto \bar{x} = J_0^\top \bar{y}, \end{aligned} \quad (22)$$

where x and $\bar{x}$ are vectors in the model space, while y and $\bar{y}$ are vectors in the data space. Both `modelling_b()` and `modelling_d()` are created with the Tapenade parameters

```

234 -head 'modelling(dcal)/(vp rho qinv)'

```

because the Jacobian is the derivative of the data `dcal` with respect to the model `m` (they are functions of physical parameters `vp`, `rho` and `qinv`) according to (8). It also highlights the fact that the tangent and reverse codes are generated independently, although they are adjoint to each other mathematically. Because all variables in tangent and adjoint modes must be paired with forward mode, they are all created with the suffix `b` and `d`, as illustrated in Table 1.

Because the differentiation is automated, there are no manual injection patches, no hand-coded adjoint source, and no risk of forgetting a chain-rule term. Because `dcalb` is an explicit input, the generated reverse routine exposes a validated arbitrary-seed $J^\top q$ interface. The FWI driver seeds the weighted residual $w^2(d_{\text{cal}} - d_{\text{obs}})$ (w is offset-dependent weighting), whereas the LSM solver passes arbitrary migration and normal-equation vectors through the

same `modelling_b()` entry point. The generated code propagates this seed backward through the recording operator into the pressure adjoint and continues reversing the extraction, injection, field updates, parameter conversion, and checkpoint replay to accumulate the sensitivities in `vpb`, `rhob`, `qinvb`:

$$\text{vpb} = \frac{\partial \Phi}{\partial \text{vp}}, \text{rhob} = \frac{\partial \Phi}{\partial \text{rho}}, \text{qinvb} = \frac{\partial \Phi}{\partial \text{qinv}}. \quad (23)$$

At a fixed background this routine applies the transpose of the same Jacobian whose forward action is computed by `modelling_d()`.

4.3. Binomial checkpointing

It is well known that the time-reversed propagation of the adjoint field must be aligned with the forward field at each time step to compute the sensitivities of the misfit with respect to the model parameters. This challenging task requires additional storage and recomputation techniques. The wavefield reconstruction method (Yang, Brossier and Virieux, 2016c) and CARFS algorithm (Yang et al., 2016b) have been proposed to do it efficiently with very low memory requirements. These techniques are non-trivial to implement. Fortunately, Tapenade adopts the revolve/binomial checkpointing scheme (Griewank, 1992; Griewank and Walther, 2000) by default, which works in generic settings regardless of whether the medium is attenuative, although the computing cost and memory overhead may be higher than the methods mentioned above.

The BINOMIAL-CKP annotation in Listing 1 instructs Tapenade to use a binomial checkpoint schedule (Symes, 2007). At runtime, `nsnap` specifies the number of snapshots available to the schedule. Each checkpoint in the current generated reverse routine stores four wavefield arrays (`p`, `vz`, `vx`, `vy`) and three SLS arrays (`xi1`, `xi2`, `xi3`), all on the padded grid, together with the CPML memory fields and the time index. In two dimensions, the out-of-plane CPML fields have scalar placeholder storage. The SLS arrays are retained even when attenuation is disabled. The seismogram `dcal` is allocated separately and is not pushed in the checkpoint-save branch. This is an explicit time-memory trade-off. More snapshots can reduce replay, while fewer snapshots reduce retained wavefield state. It does not eliminate the AD runtime's storage needs, and the exact cost depends on grid size, time steps, and the schedule. The supplied driver rejects invalid Tapenade 3.16 budgets before modelling starts. The checkpoint store uses `adStack.c` from the Tapenade ADFirstAidKit.

Listing 5 shows the generated reverse routine. The function carries a `_b` companion for every active variable, following the naming convention of Table 1; the caller must zero the adjoint arrays before each independent call. The generated `modelling_b()` creates the `adBinomial` scheduler during its initialization phase. At each time step the scheduler returns one of six actions:

1. `adBinomial_save` — push the full wavefield, CPML, and attenuation state onto the checkpoint stack;
2. `adBinomial_restore` — restore the saved state from the checkpoint stack back into the active fields, bracketed by `adStack_startRepeat/endRepeat` markers;
3. `adBinomial_restore_nodiscard` — restore and keep the checkpointed state on the stack (avoids invalidating the neighbouring forward replay);
4. `adBinomial_forward` — execute one forward time step (`fdtd_update_v`, `fdtd_update_p`, `inject_source`, `extract_wavefield`) without differentiation;
5. `adBinomial_forwardAndSave` — combine action 4 with a save;
6. `adBinomial_backward` — execute the reverse-differentiated update for that time step (`fdtd_update_v_b`, `fdtd_update_p_b`, `inject_source_b`, `extract_wavefield_b`). The kernels inside the forward loop are called in reverse order with respect to the forward pass.

Actions 1–3 operate through non-differentiated (`_nodiff`) entry points so they do not produce additional derivative contributions. Only action 6 applies the adjoint of the forward step. The binomial scheduler guarantees that every time step is reversed exactly once and that the forward trajectory is available when the adjoint step executes.

```
void modelling_b(sim_t *sim, acq_t *acq, float ***p, float ***pb, float ***vz,
    float ***vzb, float ***vx, float ***vxb, float ***vy, float ***vyb,
    float ***memD1p, float ***memD1pb, ..., float **dcal, float **dcalb) {
    int it;
    int nt = sim->nt;
```

```

284 int nsnap = sim->nsnap;
285 int adCount;
286 int i;
287 int action;
288 int step;
289 /* Referenced by Tapenade's BINOMIAL-CKP directive. */
290 pushReal4Array(rhomod[0][0], sim->n123pad);
291 pushReal4Array(vpmo[0][0], sim->n123pad);
292 convert_parameters_nodiff(sim, vp, rho, qinv, vpmo, rhomod, qinvmo,
293                          kappa, buz, bux, buy);
294 it = 0;
295 adBinomial_init(nt, nsnap, 0);
296 while(adBinomial_next(&action, &step)) {
297     switch (action) {
298     case 1 : /* save */
299         pushReal4Array(p[0][0], sim->n123pad);
300         pushReal4Array(vz[0][0], sim->n123pad);
301         pushReal4Array(vx[0][0], sim->n123pad);
302         pushReal4Array(vy[0][0], sim->n123pad);
303         pushReal4Array(memD1p[0][0], 2*sim->nb*sim->n2pad*sim->n3pad);
304         /* ... remaining CPML, SLS, and dcal state ... */
305         pushInteger4(it);
306         goto label110;
307     case 2 : /* restore */
308         adStack_startRepeat();
309         popInteger4(&it);
310         popReal4Array(memD1p[0][0], 2*sim->nb*sim->n2pad*sim->n3pad);
311         popReal4Array(vy[0][0], sim->n123pad);
312         popReal4Array(vx[0][0], sim->n123pad);
313         popReal4Array(vz[0][0], sim->n123pad);
314         /* ... remaining fields ... */
315         popReal4Array(p[0][0], sim->n123pad);
316         adStack_resetRepeat();
317         adStack_endRepeat();
318         goto label110;
319     case 3 : /* restore & discard */
320         /* ... same pop sequence as case 2 without adStack markers ... */
321         goto label110;
322     case 4 : /* forward (nodiff) */
323         if (it <= nt - 1) {
324             ftdt_update_v_nodiff(sim, p, vz, vx, vy, memD1p, memD2p,
325                                memD3p, buz, bux, buy);
326             ftdt_update_p_nodiff(sim, p, vz, vx, vy, memD1vz, memD2vx,
327                                memD3vy, xi1, xi2, xi3, kappa, qinvmo);
328             inject_source_nodiff(sim, acq, p, sim->stf[it]);
329             extract_wavfield_nodiff(sim, acq, p, dcal, it);
330             it = it + 1;
331         }
332         goto label110;
333     case 5 : /* forwardAndSave: same as case 4 with additional pushes */
334     case 6 : /* backward: reverse-differentiated update */
335         extract_wavfield_b(sim, acq, p, pb, dcal, dcalb, it);
336         inject_source_b(sim, acq, p, pb, sim->stf[it]);
337         ftdt_update_p_b(sim, p, pb, vz, vzb, vx, vxb, vy, vyb,
338                        memD1vz, memD1vzb, memD2vx, memD2vxb,
339                        memD3vy, memD3vyb, xi1, xi1b, xi2, xi2b,
340                        xi3, xi3b, kappa, kappab, qinvmo, qinvmodb);
341         /* ... state restoration then ... */

```

```

342         fdt_d_update_v_b(sim, p, pb, vz, vzb, vx, vxb, vy, vyb,
343                         memD1p, memD1pb, memD2p, memD2pb, memD3p,
344                         memD3pb, buz, buz, buz, buz, buy, buyb);
345         goto label110;
346     }
347     label110: ;
348 }
349 popReal4Array(vpmod[0][0], sim->n123pad);
350 popReal4Array(rhomod[0][0], sim->n123pad);
351 convert_parameters_b(sim, vp, vpb, rho, rhob, qinv, qinvb, vpmod, vpmodb,
352                     rhomod, rhomodb, qinvmod, qinvmodb, kappa, kappab,
353                     buz, buz, buz, buz, buy, buyb);
354 }
355

```

Listing 5: Tapenade generated adjoint routine (abridged)

4.4. Linearised inversion assembly: LSRTM and Hessian-vector product

The preceding sections showed that Tapenade generates the Jacobian action J_0 (via `modelling_d()`) and its transpose J_0^T (via `modelling_b()`) from a single forward source. To assemble these building blocks into a usable linearised inversion engine, the file `do_lsm.c` provides two symmetric wrapper routines that handle parameter conversion between logarithmic and physical spaces, data weighting, and shot stacking over MPI.

Tangent wrapper (Born modelling). The routine `lsm_tlm()` receives a model-space direction x in logarithmic units. The helper `lsm_set_direction()` converts it to physical perturbations by multiplying each slab by the corresponding background field; under the coupled $(\ln v_p, \ln I_p)$ parameterisation it also deposits a $-\rho \delta \ln v_p$ contribution into ρh . The seeded physical arrays are passed to `modelling_d()`, which returns the scattered data $d_{\text{cald}} = J_0 \delta m_{\text{phys}}$. Weighting by w_{dat} yields the weighted Born prediction

$$d_{\text{out}} = W J_0 x. \quad (24)$$

Adjoint wrapper (migration). The routine `lsm_adj()` accepts a weighted data residual, writes it into the seed array `dcalb`, and calls `modelling_b()`. The reverse sweep accumulates physical sensitivities (`vpb`, `rhob`, `qinvb`) which are then mapped back to logarithmic units by `lsm_map_adjoint()`. An `MPI_Allreduce` stacks contributions over shots:

$$g = \sum_s J_{0,s}^T W^T r_s. \quad (25)$$

where s is the source index to be summed over by MPI reduction. Listing 6 shows both wrappers.

```

362 static void lsm_tlm(sim_t *sim, acq_t *acq, int family, int npar,
363                  const int *idxpar, const float *x,
364                  workspace_t *w, float **dout)
365 {
366     int irec, it;
367     lsm_set_direction(sim, family, npar, idxpar, x, w);
368     fdt_d_null(sim, 0);
369     modelling_d(sim, acq, sim->p, w->pd, sim->vz, w->vzd,
370              sim->vx, w->vxd, sim->vy, w->vyd,
371              /* CPML, SLS, and model arrays ... */
372              sim->vp, w->vpd, sim->rho, w->rhod,
373              sim->qinv, w->qinvd, /* ... */
374              sim->dcal, w->dcald); // J * x
375     for(irec=0; irec<acq->nrec; irec++)
376         for(it=0; it<sim->nt; it++)
377             dout[irec][it] = acq->wdat[irec][it] // W J x
378                             * w->dcald[irec][it];
379 }
380

```

```

381
382 static void lsm_adj(sim_t *sim, acq_t *acq, float **residual,
383                   int family, int npar,
384                   const int *idxpar, workspace_t *w,
385                   float *g, float *greduced)
386 {
387     int irec, it;
388     for(irec=0; irec<acq->nrec; irec++)
389         for(it=0; it<sim->nt; it++)
390             w->dcalb[irec][it] = acq->wdat[irec][it] // seed = W^T r
391                             * residual[irec][it];
392     memset(w->vpb[0][0], 0, ...);
393     fdtd_null(sim, 0); fdtd_null(sim, 1);
394     modelling_b(sim, acq, sim->p, sim->pb, /* ... */
395               sim->vp, w->vpb, sim->rho, w->rhob,
396               sim->qinv, w->qinvb, /* ... */
397               sim->dcal, w->dcalb); // J^T W^T r
398     lsm_map_adjoint(sim, family, npar, idxpar, w, g);
399     MPI_Allreduce(g, greduced, ..., MPI_SUM, ...); // sum shots
400     memcpy(g, greduced, ...);
401 }
402

```

Listing 6: Symmetric wrapper pair for Born modelling and migration.

Gauss–Newton Hessian-vector product. The same building blocks supply the Gauss–Newton Hessian action in equation (15). Listing 7 shows the function `lsm_hessian_vector()`, which chains `lsm_tlm()` and `lsm_adj()` to produce

$$H_{GN}x = J^T W^T W J x, \quad (26)$$

where J is evaluated at the current FWI model m_k rather than a fixed background and no residual-curvature term appears (standard Gauss–Newton approximation). In the FWI driver `fg_fwi.c`, this function is wrapped by a bathymetry projection and a scaling factor inside `Hv_fwi()`, providing the matrix-free operator for the inner CG solve of the truncated Newton method (Section 3.3).

```

407
408 void lsm_hessian_vector(sim_t *sim, acq_t *acq, int family,
409                       int npar, const int *idxpar,
410                       const float *v, float *Hv)
411 {
412     workspace_t w;
413     float **dsyn = alloc2float(sim->nt, acq->nrec);
414     float *greduced = alloc1float(npar * sim->n123);
415
416     lsm_alloc_workspace(sim, acq, &w);
417     cpml_init(sim); extend_model_init(sim);
418     fdtd_init(sim, 0); fdtd_init(sim, 1);
419
420     lsm_tlm(sim, acq, family, npar, idxpar, v, &w, dsyn); // WJ v
421     lsm_adj(sim, acq, dsyn, family, npar, idxpar,
422            &w, Hv, greduced); // J^T W^T WJ v
423
424     fdtd_free(sim, 1); fdtd_free(sim, 0);
425     extend_model_free(sim); cpml_free(sim);
426     lsm_free_workspace(&w);
427     free2float(dsyn); free1float(greduced);
428 }
429

```

Listing 7: Gauss–Newton Hessian-vector product via chained TLM and adjoint.

Thus the two generated routines—`modelling_d()` and `modelling_b()`—supply the complete linearised inversion toolbox: Born modelling, migration, and the Gauss–Newton normal operator, all accessed through the same automated differentiation pipeline without any hand-derived adjoint stencils.

5. Gradient validation

We compare the adjoint gradient of the discrete weighted least-squares misfit with centered finite differences in each inversion-parameter class. Both parameter families are tested with all three fields present: $(\ln v_p, \ln \rho, \ln Q^{-1})$ and $(\ln v_p, \ln I_p, \ln Q^{-1})$, where $I_p = \rho v_p$. For a direction d confined to one parameter class, the two estimates of the directional derivative are

$$g_{AD} = \langle \nabla_m \Phi(m), d \rangle, \quad g_{FD}(h) = \frac{\Phi(m + hd) - \Phi(m - hd)}{2h}. \quad (27)$$

We measure their relative difference using the fixed adjoint magnitude:

$$e_{FD}(h) = \frac{|g_{FD}(h) - g_{AD}|}{|g_{AD}|}. \quad (28)$$

All tested directions have nonzero g_{AD} . The centered formula has an $O(h^2)$ truncation error, so halving h should reduce this difference by approximately four before floating-point error dominates. The normalization is independent of h and therefore preserves the convergence order of the absolute derivative difference. These directional tests cover every parameter class and include the parameter transformation and MPI reduction.

The reproducible experiment uses an 81×81 grid with 5 m spacing, 20 CPML layers, eighth-order spatial stencils, 800 time steps of 0.5 ms, and ten binomial checkpoints. Sources at $(x, z) = (125, 100)$ and $(275, 100)$ m are assigned to two MPI ranks. Each shot has 17 receivers at $z = 50$ m and $x = 40, 60, \dots, 360$ m. The source is a 25-Hz Ricker wavelet with a 60-ms delay and amplitude 10^6 . All cases use three SLS mechanisms fitted for $Q = 100$ over 1–40 Hz. The starting fields are $v_p = 2000 + 0.5z$ m/s, $\rho = 1800 + 0.3z$ kg/m³, and $Q^{-1} = 0.01$. Synthetic observations are regenerated with these fields multiplied by $1 + 0.08A$, $1 + 0.06A$, and $1 + 0.5A$, respectively, where

$$A(x, z) = \exp \left[- \left(\frac{x - 210}{75} \right)^2 - \left(\frac{z - 200}{65} \right)^2 \right].$$

CPML coefficients use a fixed reference velocity of 2500 m/s and reference frequency of 15 Hz for every evaluation. Keeping these passive coefficients independent of the model avoids an unaccounted derivative through the model-dependent maximum velocity.

For each of the six parameter cases, two deterministic random-sign directions (seeds 12345 and 67890) perturb all 6480 active cells in that class. The top row is excluded by the bathymetry mask. In particular, the family-2 velocity test holds I_p fixed, so that its gradient includes both the velocity and density chain-rule contributions. The adjoint is evaluated once per direction, without preconditioning; subsequent perturbed objectives require only forward solves. Misfits are accumulated and reduced in double precision; model arrays, wavefields, and gradients remain single precision. Nine step sizes, obtained by eight successive halvings, start from $h = 1/16$ for velocity, density, and impedance, and from $h = 1/2$ for inverse quality factor. Objectives and directional derivatives are saved with 17-digit output.

Table 2 reports least-squares slopes of $\log e_{FD}$ against $\log h$, fitted over four fixed interior steps: $[1/256, 1/32]$ for velocity, density, and impedance, and $[1/32, 1/4]$ for attenuation. The same step indices are used for every case, without an automatic fit-window search. Each directional check requires a slope between 1.8 and 2.2 and a minimum relative difference below 1% over the complete sweep.

All twelve comparisons satisfy these criteria. As shown in Figures 1 and 2, the fitted slopes range from 1.917 to 2.030, consistent with the second-order centered formula. In particular, velocity at fixed impedance gives slopes of 1.999–2.005, confirming the coupled velocity–density chain rule. The minimum relative differences across the twelve sweeps range from 1.21×10^{-5} to 3.13×10^{-4} ; every direction therefore reaches agreement within 0.032%. These minima can occur at different step sizes. At smaller steps, single-precision model and wavefield arithmetic limits further improvement despite double-precision misfit accumulation.

Table 2

Centered finite-difference versus adjoint comparison for all six parameter cases. Slope intervals span the two random directions. The final column gives the range of minimum relative differences attained separately by the two directions.

Parameter	Step interval	FD error slopes	Best FD errors
$\ln v_p$ (family 1)	0.00390625–0.03125	1.998–2.009	1.13×10^{-4} – 1.17×10^{-4}
$\ln \rho$ (family 1)	0.00390625–0.03125	2.002–2.030	1.21×10^{-5} – 3.17×10^{-5}
$\ln Q^{-1}$ (family 1)	0.03125–0.25	1.917–2.021	9.89×10^{-5} – 2.46×10^{-4}
$\ln v_p$ (family 2)	0.00390625–0.03125	1.999–2.005	2.29×10^{-5} – 3.13×10^{-4}
$\ln I_p$ (family 2)	0.00390625–0.03125	1.981–1.998	2.38×10^{-5} – 1.83×10^{-4}
$\ln Q^{-1}$ (family 2)	0.03125–0.25	1.986–1.996	2.61×10^{-5} – 6.96×10^{-5}

6. Multiparameter FWI of the Marmousi model

We use the two-dimensional Marmousi model to assess three aspects of the workflow: the cost of a gradient evaluation, l-BFGS convergence with generated and manually coded adjoints, and the benefit of incorporating Gauss–Newton curvature. The common grid has $n_z = 151$ and $n_x = 461$ points with $\Delta z = \Delta x = 20$ m, spanning 3.0 km in depth and 9.2 km horizontally. For the inversion runs, the acquisition comprises 24 sources evenly distributed from $x = 100$ to 9100 m at 5 m depth and 461 receivers at 10 m depth, spaced every 20 m across the model. A 5 Hz Ricker wavelet is sampled at 2 ms for 2500 time steps (5 s). The forward calculation uses fourth-order spatial finite differences, a free surface, and a 20-cell absorbing boundary. All Marmousi experiments use the acoustic limit of the modelling engine (`nrelax=0`) (since viscoacoustic physics is not feasible in original SMIwiz implementation for comparison).

The initial velocity and density are Gaussian-smoothed versions of the reference fields, with a standard deviation of four grid cells (80 m). The active parameters are $(\ln v_p, \ln \rho)$. All three inversion runs share these models, acquisition, wavelet, observations, and data weights. They use no preconditioning and allow at most 50 outer updates, with an initial unit trial step, at most ten line-search trials per update, and a relative objective tolerance of 10^{-8} . Both l-BFGS runs store five correction pairs; both Tapenade-based runs 50 iterations using 20 binomial checkpoints. Convergence is measured by Φ_k/Φ_0 and $\|g_k\|_2/\|g_0\|_2$, normalized separately for each run. All three runs exhaust the update budget without reaching the objective tolerance.

Figure 3 shows the reference and initial models together with the result of the Tapenade-based l-BFGS inversion. The velocity inversion sharpens dipping reflectors and recovers much of the laterally varying structure lost through smoothing. Its root-mean-square error over the full grid decreases from 250.3 to 169.3 m/s. Density recovery is less consistent: although some contrasts are sharpened, the final model contains artifacts and its full-grid root-mean-square error increases from 71.2 to 76.5 kg/m³. These contrasting outcomes provide context for interpreting the convergence comparisons below.

6.1. Runtime and memory cost

The manual adjoint combines a time-dependent computing box (Yang, 2024) with backward wavefield reconstruction from saved boundaries (Yang et al., 2016c). Tapenade instead advances the full padded domain and replays forward steps under a binomial checkpointing schedule. These different strategies affect both runtime and memory use.

To measure this cost, we evaluate the objective and velocity–density gradient once for the first shot, without optimization iterations. The benchmark records the memory and runtime for 500, 1000, 1500, and 2500 time steps. The manual method saves boundaries at every time step. Tapenade uses 20 checkpoints in the record-length comparison; a separate sweep uses 5, 10, 20, and 40 checkpoints at 2500 steps.

Both implementations use identical compiler settings and run on the same CPU core of an Intel Core i5-12500H, with one MPI process and one OpenMP thread. We measure total process wall time, including initialization, input/output, the initial forward solve, and all replay, together with peak resident memory. Figure 4 reports medians and observed ranges from three runs after a discarded warmup. The predicted records agree to within 8.65×10^{-7} in relative L_2 error, supporting the consistency of the forward calculations.

At 2500 time steps, the manual method requires 8.44 s and 101.0 MiB, whereas Tapenade with 20 checkpoints requires 36.35 s and 123.5 MiB: 4.31 times the runtime and 22% more memory. With five checkpoints, Tapenade uses only 81.9 MiB but takes 48.77 s; increasing the budget to 40 checkpoints reduces runtime to 34.70 s while

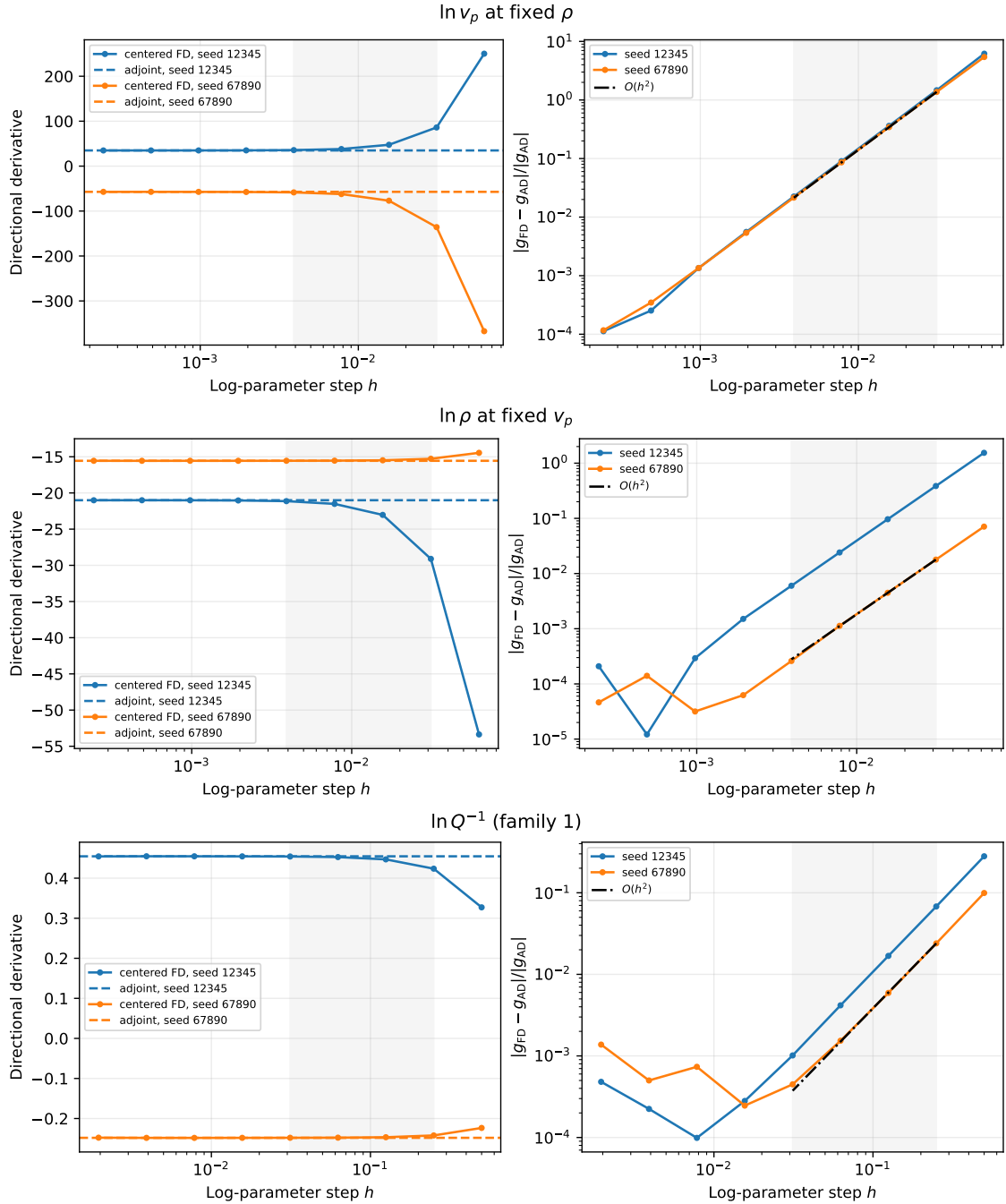


Figure 1: Centered finite-difference and adjoint directional derivatives for family 1: velocity, density, and inverse quality factor. Left: the finite-difference estimates approach the constant adjoint values as the step decreases. Right: their relative differences with an $O(h^2)$ reference line. Colors distinguish the two directions; shaded intervals indicate the fixed slope-fit range.

raising memory use to 179.1 MiB. The AD gradient is unchanged across these budgets. Thus, the runtime penalty is accompanied by a tunable memory requirement, which can fall below or exceed that of the manual method.

The manual boundary history grows with record length, whereas a fixed checkpoint budget bounds the stored forward states; receiver-data storage still grows in both methods. The Taped implementation also retains attenuation

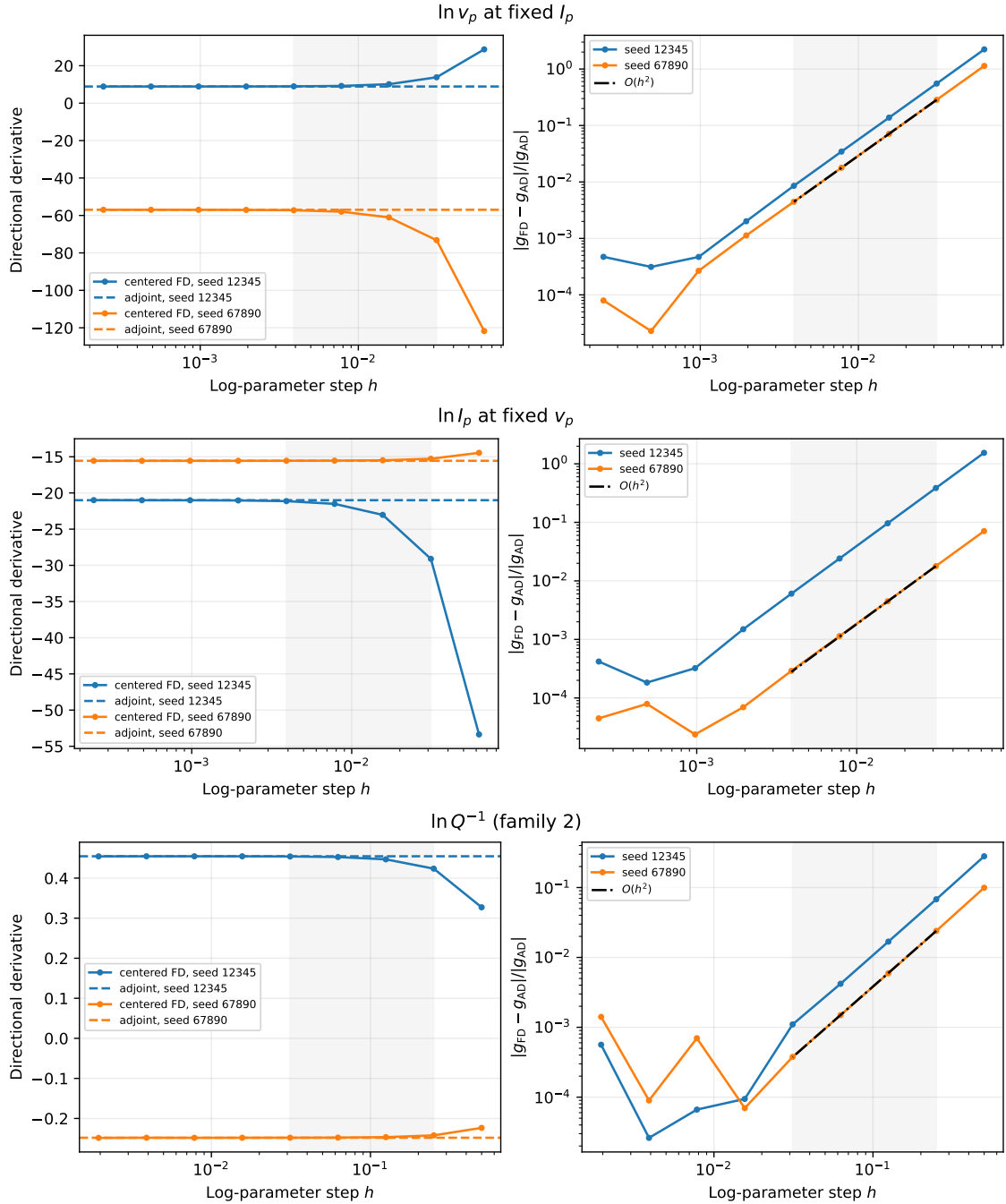


Figure 2: Centered finite-difference and adjoint comparisons for family 2, with the same conventions as Figure 1. The velocity perturbation holds impedance fixed; the impedance perturbation holds velocity fixed. Attenuation is also checked with all three parameter fields present.

memory variables even in the acoustic case. The measurements therefore include the combined effects of storage, replay, and computational-domain restriction. We remind that these statics should not be over interpreted, as they fully depend on the size of the model (which can be drastic in 3D).

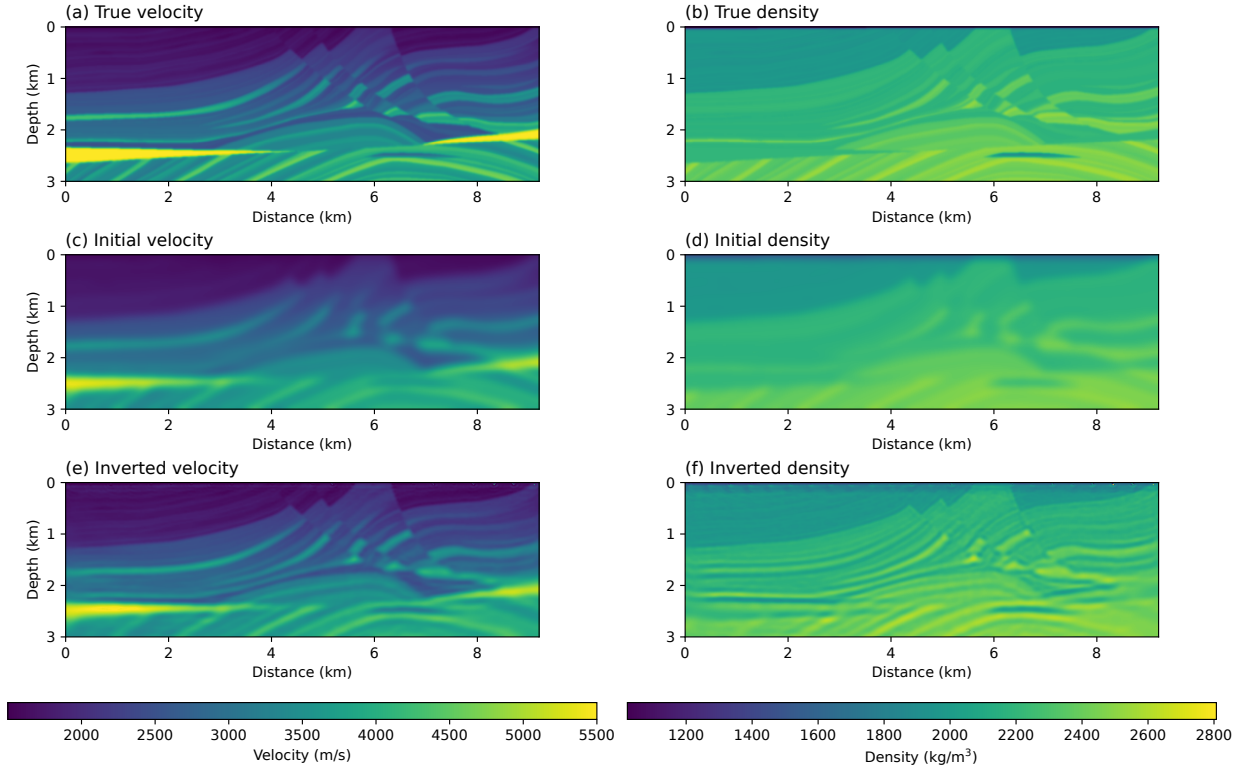


Figure 3: Marmousi FWI using the Tapenade-generated adjoint and I-BFGS: true (top row), initial (middle row), and inverted models after 50 updates (bottom row). Velocity is shown in the left column, panels (a), (c), and (e); density is shown in the right column, panels (b), (d), and (f). Each column uses a common colour scale.

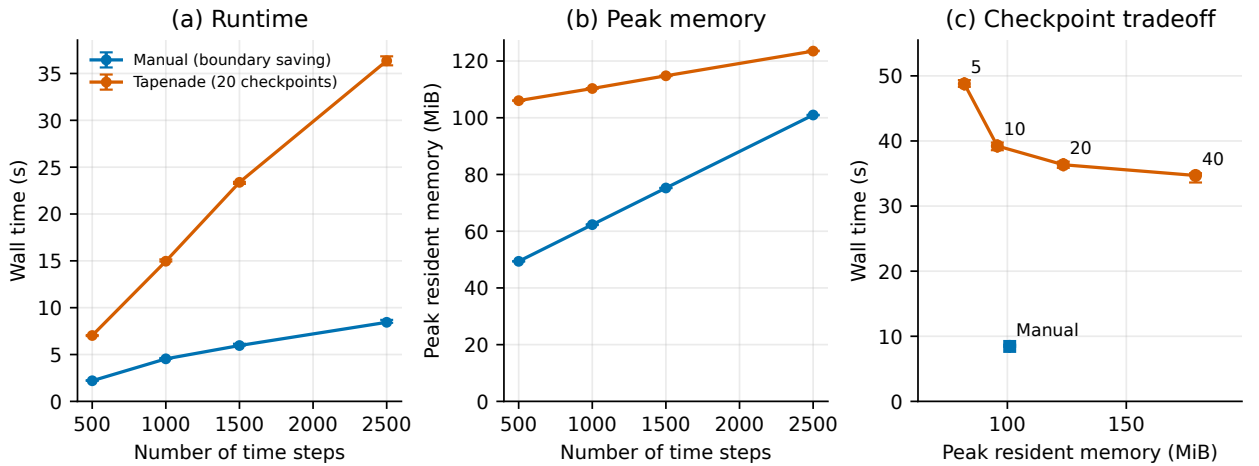


Figure 4: Cost of one acoustic objective and gradient evaluation on the 151 × 461 Marmousi grid. (a) Wall time and (b) peak resident memory versus number of time steps, using 20 AD checkpoints and manual boundary saving at every step. (c) Memory–runtime tradeoff for 5, 10, 20, and 40 checkpoints at 2500 steps. Points show medians of three runs after one discarded warmup; error bars span the observed minima and maxima.

6.2. Comparison between DTA (Tapenade) and ATD (SMIwiz)

With the common inversion setup fixed, we first compare l-BFGS using the DTA approach (Tapenade-generated adjoint) with l-BFGS using ATD method (the original SMIwiz manual adjoint). This comparison measures the progress of the two implementations under the same optimizer and update budget.

Figure 5 compares the objective and gradient histories. Normalization removes the different constant scalings used by the implementations: the original objective includes Δt , and each code also applies a scale determined by its initial gradient. Both runs reduce the objective monotonically, with Tapenade achieving a lower normalized objective at every recorded update after the initial state. At update 20, the normalized objectives are 1.16×10^{-1} for original SMIwiz and 1.00×10^{-1} for Tapenade. After 50 updates, they reach 3.12×10^{-2} and 2.20×10^{-2} , respectively, corresponding to reductions of 96.9% and 97.8% from their initial values. Tapenade's final normalized objective is approximately 29.6% lower; it first falls below the original implementation's final value at update 41. The gradient norms fluctuate in both runs, ending at 7.38×10^{-2} and 6.45×10^{-2} of their respective initial norms. Each run requires 52 objective/gradient evaluations, including the initial evaluation and one line-search retry. Thus the Tapenade-based implementation reduces the normalized misfit faster per update slightly, thanks to the accuracy of DTA approach used by AD.

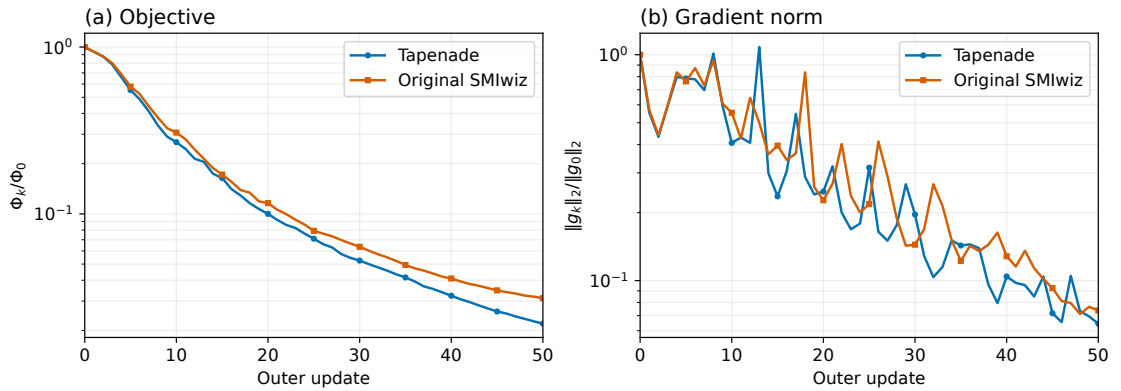


Figure 5: Convergence of original SMIwiz and the Tapenade-based implementation for the same Marmousi velocity–density inversion using l-BFGS: (a) normalized objective and (b) normalized gradient norm versus outer update, including the final states after 50 updates.

6.3. Comparison between Gauss–Newton and l-BFGS

We next retain the Tapenade-generated operators and replace l-BFGS with Gauss–Newton, reusing the preceding l-BFGS run as the baseline. The new search direction incorporates approximate curvature through the Gauss–Newton Hessian. The method approximately solves $H_{GN} \delta m = -g$ using at most five inner CG iterations and a relative residual tolerance of 0.01. Each matrix-free Hessian-vector product uses the tangent and adjoint routines described in Section 4.2.

Figure 6 shows a further improvement in convergence per outer update. Gauss–Newton reduces the normalized objective to 2.24×10^{-2} at update 20 and 2.13×10^{-2} at update 21, passing the final l-BFGS value in fewer than half as many updates. At update 50, its normalized objective is 8.07×10^{-3} , approximately 2.7 times lower than the l-BFGS result. Its gradient norm fluctuates but decreases overall to 2.69×10^{-2} of its initial value.

The advantage per outer update must be distinguished from computational cost. Gauss–Newton accepts unit steps without line-search retries and requires 51 objective/gradient evaluations, including the initial one. This count excludes the Hessian-vector products in the inner CG solves, each of which requires tangent and adjoint propagation. The reduced number of outer updates therefore does not establish a runtime advantage.

7. Discussions

We have picked up the simplest possible example to demonstrate how to construct seismic imaging codes with Tapenade. However, the computation efficiency may be dramatically different from the original SMIwiz implementation, due to completely different strategy in building up the inversion gradient. The smart computing box to limit the

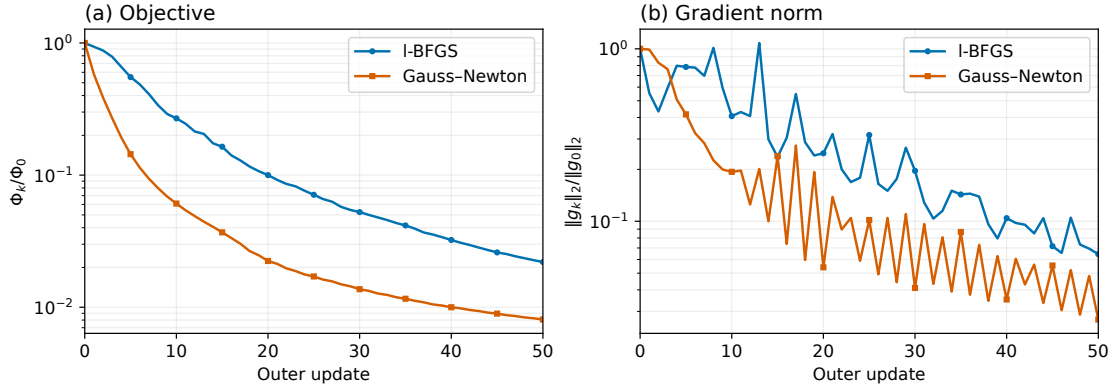


Figure 6: Convergence of I-BFGS and Gauss-Newton for the same Marmousi velocity-density inversion: (a) normalized objective and (b) normalized gradient norm versus outer update. Gauss-Newton uses at most five inner CG iterations per outer update. The I-BFGS curve is the same Tapenade baseline as in Figure 5.

modelling domain at each time step (Yang, 2024), as well as the efficient backward wavefield reconstruction via boundary saving (Yang et al., 2016c), are the missing elements in this Tapenade-based workflow.

The Marmousi example illustrates a limitation of multiparameter inversion: a substantial reduction in data misfit can coexist with density artifacts. Although I-BFGS and Gauss-Newton both incorporate approximate curvature information, their use alone does not guarantee resolution of parameter ambiguity. The convergence comparison in Section 6.3 shows that Gauss-Newton reduces the data misfit faster per outer update, with additional work in the inner CG solves. This comparison of objective and gradient histories alone does not establish which optimizer better recovers each parameter field. At the same data fitting level, the outcomes of many I-BFGS and Gauss-Newton iterations through our implementation are highly similar, which is highly consistent with Yang et al. (2018).

We highlight that the above demonstration is based on finite difference discretization of viscoacoustic wave equation. The reconstruction of seismic imaging codes was done using C programming. However, the proposed workflow by Tapenade is rather generic, applicable to Fortran programming equally. It is independent of the numerical discretization method used, independent of the wave physics (with or without anisotropy or seismic attenuation). Despite a 2D brief presentation, the code built from Tapenade allows the reproduction of all data-domain seismic imaging examples (3D and real data) published in SMIwiz package (Yang, 2024; Yang and Ji, 2026), since the modelling engine is directly extracted from SMIwiz and therefore capable of simulating in both 2D and 3D cases. We have also verified this workflow for 2D elastic imaging based on the spectral element method coded in C. It is not to presented it here to simplify the exposition of Tapenade workflow.

8. Conclusions

Manual adjoint coding for multiparameter FWI and LSRTM is tedious and error-prone. This paper presented an automated alternative built on a single Tapenade differentiation boundary around a forward modelling routine, together with the checkpoint runtime, post-processing fix, and validation suite needed for production use. The forward routine uses explicit state arguments so that Tapenade distinguishes active fields from passive configuration; the receiver recording stays inside the differentiated boundary so the generated reverse program includes the adjoint of the sampling operator without manual patching. From this one source, Tapenade independently produces the tangent linear model (`modelling_d()`) and the adjoint model (`modelling_b()`), supplying both J_0 and J_0^T for Born modelling, migration, and matrix-free Gauss-Newton products. Binomial checkpointing manages the forward-trajectory storage, and a version-controlled fix script reconciles the generated scalar stack operations with contiguous C array storage without altering the automatically derived reverse physics. Directional finite-difference tests exhibit the expected second-order agreement for all inversion parameters. On the acoustic Marmousi problem, the generated adjoint improves I-BFGS convergence per update relative to the original manual implementation, and reuse of the linearised operators enables a further improvement with Gauss-Newton. These gains come with additional computational work: the generated adjoint

requires replay, and Gauss–Newton requires inner linear solves. The checkpoint budget makes memory use adjustable, with the measured requirement falling below or exceeding that of the manual method.

Acknowledgement

I thank William Symes for introducing me to the Tapenade AD tool in 2015. The limited use of source-transformation AD in large-scale seismic-imaging software motivated me to promote this practical workflow.

Computer Code Availability

The code has been released publicly via github repository https://github.com/yangpl/SMIwiz_visco.

A. Discrete forward time stepping as recurrence

Substituting the velocity update (17a) into (17b) expresses the memory variables at level $n + 1$,

$$\xi_\ell^{n+1} = (1 - e^{-\omega_\ell \Delta t}) \nabla \cdot \mathbf{v}^{n-\frac{1}{2}} - (1 - e^{-\omega_\ell \Delta t}) \Delta t \nabla \cdot \rho^{-1} \nabla p^n + e^{-\omega_\ell \Delta t} \xi_\ell^n. \quad (29)$$

so that

$$\xi_\ell^n + \xi_\ell^{n+1} = (1 + e^{-\omega_\ell \Delta t}) \xi_\ell^n + (1 - e^{-\omega_\ell \Delta t}) \nabla \cdot \mathbf{v}^{n-\frac{1}{2}} - (1 - e^{-\omega_\ell \Delta t}) \Delta t \nabla \cdot \rho^{-1} \nabla p^n. \quad (30)$$

For the pressure row, substituting (17a) into (17c) with (30) gives

$$p^{n+1} = p^n - \Delta t \kappa \nabla \cdot \mathbf{v}^{n-\frac{1}{2}} + \Delta t^2 \kappa \nabla \cdot \rho^{-1} \nabla p^n + \frac{\Delta t}{2} \kappa Q^{-1} \sum_{\ell=1}^L y_\ell \left(\xi_\ell^n + \xi_\ell^{n+1} \right) \quad (31)$$

$$\begin{aligned} &= \left(-\Delta t \kappa \nabla \cdot + \frac{\Delta t}{2} \kappa Q^{-1} \sum_{\ell=1}^L y_\ell (1 - e^{-\omega_\ell \Delta t}) \nabla \cdot \right) \mathbf{v}^{n-\frac{1}{2}} \\ &\quad + \left(1 + \Delta t^2 \kappa \nabla \cdot \rho^{-1} \nabla - \frac{\Delta t^2}{2} \kappa Q^{-1} \sum_{\ell=1}^L y_\ell (1 - e^{-\omega_\ell \Delta t}) \nabla \cdot \rho^{-1} \nabla \right) p^n \\ &\quad + \frac{\Delta t}{2} \kappa Q^{-1} \sum_{\ell=1}^L y_\ell (1 + e^{-\omega_\ell \Delta t}) \xi_\ell^n, \end{aligned} \quad (32)$$

Equations (17a), (17b) and (32) give a single one-step recurrence for the source-free interior update. Keeping the state ordering $(\mathbf{v}, \xi, p)$ used in Section 2, define

$$u^n = \begin{pmatrix} \mathbf{v}^{n-\frac{1}{2}} \\ \xi^n \\ p^n \end{pmatrix}. \quad (33)$$

Then

$$u^{n+1} = L u^n, \quad L = \begin{pmatrix} I & \mathbf{0} & L_{vp} \\ L_{\xi v} & L_{\xi \xi} & L_{\xi p} \\ L_{pv} & L_{p \xi} & L_{pp} \end{pmatrix} \quad (34)$$

where the non-zero blocks of the evolution matrix L read

$$L_{vp} = -\Delta t \rho^{-1} \nabla, \quad (35a)$$

$$L_{pv} = -\Delta t \kappa \nabla \cdot + \frac{\Delta t}{2} \kappa Q^{-1} \sum_{\ell=1}^L y_\ell (1 - e^{-\omega_\ell \Delta t}) \nabla \cdot, \quad (35b)$$

$$L_{pp} = 1 + \Delta t^2 \kappa \nabla \cdot \rho^{-1} \nabla - \frac{\Delta t^2}{2} \kappa Q^{-1} \sum_{\ell=1}^L y_{\ell} (1 - e^{-\omega_{\ell} \Delta t}) \nabla \cdot \rho^{-1} \nabla, \quad (35c)$$

$$L_{p\xi} = \frac{\Delta t}{2} \kappa Q^{-1} \text{row}_{\ell} (y_{\ell} (1 + e^{-\omega_{\ell} \Delta t}) I), \quad (35d)$$

$$L_{\xi v} = \text{col}_{\ell} ((1 - e^{-\omega_{\ell} \Delta t}) \nabla \cdot), \quad (35e)$$

$$L_{\xi p} = -\text{col}_{\ell} ((1 - e^{-\omega_{\ell} \Delta t}) \Delta t \nabla \cdot \rho^{-1} \nabla), \quad (35f)$$

$$L_{\xi\xi} = \text{blkdiag}_{\ell} (e^{-\omega_{\ell} \Delta t} I), \quad (35g)$$

where row_{ℓ} concatenates the L blocks horizontally, col_{ℓ} stacks them vertically, and blkdiag_{ℓ} places them on the block diagonal. The recurrence (34) makes explicit that each time step is a linear map of the state. CPML introduces additional memory fields outside this interior recurrence.

References

- Chavent, G., 2009. Nonlinear least squares for inverse problems. Springer Dordrecht Heidelberg London New York.
- Chavent, G., Plessix, R.E., 1999. An optimal true-amplitude least-squares prestack depth-migration operator. *Geophysics* 64, 508–515.
- Farmer, P., Zhou, Z.Z., Jones, D., 2009. The role of reverse time migration in imaging and model estimation. *The Leading Edge* 28, 436–441.
- Gaikwad, S.S., Narayanan, S.H.K., Hascoët, L., Campin, J.M., Pillar, H., Nguyen, A., Hückelheim, J., Hovland, P., Heimbach, P., 2025. MITgcm-AD v2: Open source tangent linear and adjoint modeling framework for the oceans and atmosphere enabled by the automatic differentiation tool Tapenade. *Future Generation Computer Systems* 163, 107512. doi:10.1016/j.future.2024.107512.
- Griewank, A., 1992. Achieving logarithmic growth of temporal and spatial complexity in reverse automatic differentiation. *Optimization Methods and Software* 1, 35–54.
- Griewank, A., Walther, A., 2000. Algorithm 799: Revolve: An implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software* 26, 19–45.
- Griewank, A., Walther, A., 2008. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. 2 ed., SIAM.
- Hascoët, L., Pascual, V., 2013. The Tapenade automatic differentiation tool: Principles, model, and specification. *ACM Transactions on Mathematical Software* 39, 20:1–20:43.
- Komatitsch, D., Martin, R., 2007. An unsplit convolutional perfectly matched layer improved at grazing incidence for the seismic wave equation. *Geophysics* 72, SM155–SM167.
- Liu, F., Li, H., Zou, G., Li, J., 2025. Automatic differentiation-based full waveform inversion with flexible workflows. *Journal of Geophysical Research: Machine Learning and Computation* 2, e2024JH000542. doi:10.1029/2024JH000542.
- Nemeth, T., Wu, C., Schuster, G.T., 1999. Least-squares migration of incomplete reflection data. *Geophysics* 64, 208–221.
- Pratt, R.G., Shin, C., Hicks, G.J., 1998. Gauss–newton and full Newton methods in frequency-space seismic waveform inversion. *Geophysical Journal International* 133, 341–362. doi:10.1046/j.1365-246X.1998.00498.x.
- Richardson, A., 2018. Seismic full-waveform inversion using deep learning tools and techniques. doi:10.48550/arXiv.1801.07232, arXiv:1801.07232.
- Santosa, F., Symes, W.W., 1988. Computation of the hessian for least-squares solutions of inverse problems of reflection seismology. *Inverse problems* 4, 211.
- Sirkes, Z., Tziperman, E., 1997. Finite difference of adjoint or adjoint of finite difference? *Monthly Weather Review* 125, 3373–3378. doi:10.1175/1520-0493(1997)125<3373:FD0A0A>2.0.CO;2.
- Symes, W.W., 2007. Reverse time migration with optimal checkpointing. *Geophysics* 72, SM213–SM221. doi:10.1190/1.2742686.
- Tarantola, A., 1984. Linearized inversion of seismic reflection data. *Geophysical prospecting* 32, 998–1015.
- Vigh, D., Jiao, K., Watts, D., Sun, D., 2014. Elastic full-waveform inversion application using multicomponent measurements of seismic data collection. *Geophysics* 79, R63–R77. doi:10.1190/geo2013-0055.1.
- Virieux, J., Operto, S., 2009. An overview of full-waveform inversion in exploration geophysics. *Geophysics* 74, WCC1–WCC26.
- Yang, P., 2024. SMIwiz: An integrated toolbox for multidimensional seismic modelling and imaging. *Computer Physics Communications* 295, 109011. URL: <https://www.sciencedirect.com/science/article/pii/S0010465523003569>, doi:10.1016/j.cpc.2023.109011.
- Yang, P., Brossier, R., Métivier, L., Virieux, J., 2016a. A review on the systematic formulation of 3D multiparameter full waveform inversion in viscoelastic medium. *Geophysical Journal International* 207, 129–149. doi:10.1093/gji/ggw262.
- Yang, P., Brossier, R., Métivier, L., Virieux, J., 2016b. Wavefield reconstruction in attenuating media: A checkpointing-assisted reverse-forward simulation method. *Geophysics* 81, R349–R362. doi:10.1190/geo2016-0082.1.
- Yang, P., Brossier, R., Métivier, L., Virieux, J., Zhou, W., 2018. A Time-Domain Preconditioned Truncated Newton Approach to Visco-acoustic Multiparameter Full Waveform Inversion. *SIAM Journal on Scientific Computing* 40, B1101–B1130.
- Yang, P., Brossier, R., Virieux, J., 2016c. Wavefield reconstruction by interpolating significantly decimated boundaries. *Geophysics* 81, T197–T209. doi:10.1190/geo2015-0711.1.
- Yang, P., Ji, Z., 2026. A comparative study of data- and image-domain LSRTM under velocity–impedance parametrization. *Computers & Geosciences* 208, 106091. URL: <https://www.sciencedirect.com/science/article/pii/S0098300425002419>, doi:10.1016/j.cageo.2025.106091.

624 Zhu, W., Xu, K., Darve, E., Beroza, G.C., 2021. A general approach to seismic inversion with automatic differentiation. *Computers & Geosciences*
625 151, 104751. doi:10.1016/j.cageo.2021.104751.